

IF25-22017

PENGEMBANGAN APLIKASI MOBILE

PERTEMUAN 5

Navigasi Antar Layar

Multi-Screen Navigation di Compose Multiplatform

Program Studi Teknik Informatika

Institut Teknologi Sumatera

Tahun Akademik 2024/2025

[Logo
Kemendikbud]

[Logo Diktisaintek]

[Logo ITERA]

OUTLINE PERTEMUAN

01

Review Minggu Lalu

State Management dan MVVM

02

Konsep Navigasi

Single Activity, Multiple Screens

03

Navigation Component

NavHost, NavController, Routes

04

Implementasi Navigasi

Setup dan Basic Navigation

05

Passing Arguments

Mengirim data antar screen

06

Nested Navigation

Bottom Navigation dan Drawer

CAPAIAN PEMBELAJARAN

CPMK0501

Mahasiswa mampu menerapkan konsep pemrograman untuk pengembangan perangkat lunak.

Indikator: Mampu mengimplementasikan navigasi multi-screen dalam aplikasi

Setelah pertemuan ini, mahasiswa mampu:

- ✓ Memahami konsep navigasi dalam aplikasi mobile
- ✓ Menggunakan Navigation Component di Compose Multiplatform
- ✓ Mendefinisikan routes dan destinations
- ✓ Mengirim data (arguments) antar screen
- ✓ Mengimplementasikan Bottom Navigation dan Navigation Drawer

REVIEW MINGGU LALU

Pertemuan 4: State Management dan MVVM

State

Data yang berubah dan mempengaruhi UI

remember + mutableStateOf

Menyimpan state yang survive recomposition

State Hoisting

State di parent, events naik ke atas

MVVM

Model-View-ViewModel architecture

ViewModel

Menyimpan UI state dengan StateFlow

? Pertanyaan: Bagaimana cara berpindah dari satu layar ke layar lain?

BAGIAN 1

KONSEP NAVIGASI

Berpindah antar layar dalam aplikasi

APA ITU NAVIGASI?

Navigasi adalah mekanisme yang memungkinkan pengguna berpindah antar layar (screen/destination) dalam aplikasi. Di Compose, navigasi dikelola secara deklaratif menggunakan Navigation Component.

Contoh Navigasi dalam Aplikasi:

E-Commerce

Home → Product List → Product Detail → Cart → Checkout

Social Media

Feed → Profile → Settings → Edit Profile

Banking

Login → Dashboard → Transfer → Confirmation → Receipt

Notes App

Note List → Note Detail → Edit Note → Back to List

ANALOGI: PETA DAN GPS

Bayangkan Aplikasi sebagai Kota

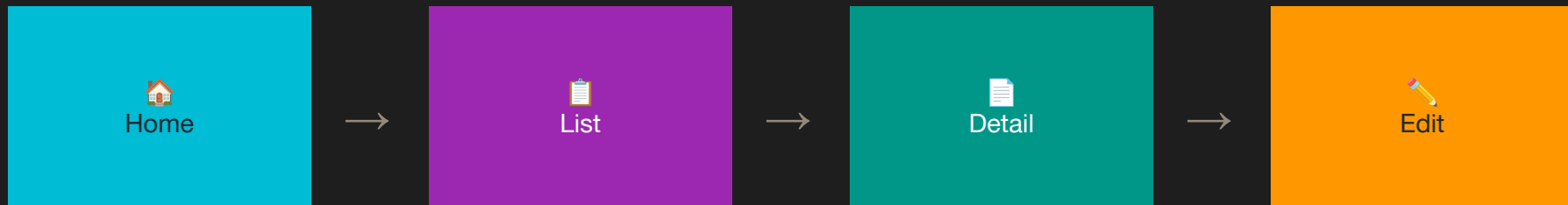
Screens = Lokasi/Tempat (rumah, kantor, mall, restoran)

Routes = Jalan yang menghubungkan lokasi

NavController = GPS yang mengatur perjalanan

Back Stack = Riwayat tempat yang sudah dikunjungi

Arguments = Barang bawaan saat pergi ke tempat tertentu



Back Stack: Home → List → Detail → Edit

← Back button akan kembali ke screen sebelumnya

KOMPONEN UTAMA NAVIGASI

NavHost

Container yang menampung semua destinations. Menentukan composable mana yang ditampilkan berdasarkan route saat ini.

NavController

Central API untuk navigasi. Mengatur perpindahan antar screen, back stack, dan state navigasi.

Routes

String unik yang mengidentifikasi setiap destination. Seperti URL path untuk setiap screen.

Struktur Dasar

```
val navController = rememberNavController() // NavController

NavHost(                                // NavHost
    navController = navController,
    startDestination = "home"           // Route awal
) {
    composable("home") { HomeScreen() } // Route: "home"
    composable("detail") { DetailScreen() } // Route: "detail"
}
```

BAGIAN 2

NAVIGATION COMPONENT

Setup dan konfigurasi navigasi

SETUP DEPENDENCY

build.gradle.kts

```
// Di file build.gradle.kts (shared module)

kotlin {
    sourceSets {
        commonMain.dependencies {
            // Navigation untuk Compose Multiplatform
            implementation("org.jetbrains.androidx.navigation:navigation-compose:2.7.0-alpha07")

            // ViewModel (jika belum ada)
            implementation("org.jetbrains.androidx.lifecycle:lifecycle-viewmodel-compose:2.8.0")
        }
    }
}

// Catatan:
// - Versi navigation-compose untuk KMP masih dalam pengembangan aktif
// - Alternatif: Voyager, Decompose, atau PreCompose
// - Untuk project sederhana, bisa juga buat navigasi manual dengan state
```

MENDEFINISIKAN ROUTES

Best Practice: Gunakan sealed class atau object untuk routes

Routes.kt

```
// Cara 1: Menggunakan sealed class (Recommended)
sealed class Screen(val route: String) {
    object Home : Screen("home")
    object NoteList : Screen("note_list")
    object NoteDetail : Screen("note_detail/{noteId}") {
        fun createRoute(noteId: Int) = "note_detail/$noteId"
    }
    object AddNote : Screen("add_note")
    object Settings : Screen("settings")
}

// Cara 2: Menggunakan object sederhana
object Routes {
    const val HOME = "home"
    const val NOTE_LIST = "note_list"
    const val NOTE_DETAIL = "note_detail/{noteId}"
    const val ADD_NOTE = "add_note"
    const val SETTINGS = "settings"
}
```

SETUP NAVHOST

AppNavigation.kt

```
@Composable
fun AppNavigation() {
    // 1. Buat NavController
    val navController = rememberNavController()

    // 2. Setup NavHost dengan semua destinations
    NavHost(
        navController = navController,
        startDestination = Screen.Home.route // Screen pertama
    ) {
        // 3. Definisikan setiap destination
        composable(Screen.Home.route) {
            HomeScreen(
                onNavigateToList = { navController.navigate(Screen.NoteList.route) }
            )
        }

        composable(Screen.NoteList.route) {
            NoteListScreen(
                onNoteClick = { noteId ->
                    navController.navigate(Screen.NoteDetail.createRoute(noteId))
                },
                onAddClick = { navController.navigate(Screen.AddNote.route) }
            )
        }

        composable(Screen.AddNote.route) {
            AddNoteScreen(onBack = { navController.popBackStack() })
        }
    }
}
```

NAVIGASI DASAR

`navigate()`

Pindah ke screen baru. Screen baru ditambahkan ke back stack.

```
navController.navigate("detail")
```

`popBackStack()`

Kembali ke screen sebelumnya. Menghapus screen saat ini dari back stack.

```
navController.popBackStack()
```

Contoh Penggunaan

```
@Composable
fun HomeScreen(onNavigateToDetail: () -> Unit) {
    Column {
        Text("Home Screen")
        Button(onClick = onNavigateToDetail) { // Navigate forward
            Text("Go to Detail")
        }
    }
}

@Composable
fun DetailScreen(onBack: () -> Unit) {
    Column {
        Text("Detail Screen")
        Button(onClick = onBack) { // Navigate back
            Text("Back to Home")
        }
    }
}
```

NAVIGATION OPTIONS

Opsi tambahan saat navigasi untuk mengontrol back stack:

Navigation Options

```
// 1. popUpTo - Hapus screens dari back stack sampai destination tertentu
navController.navigate("home") {
    popUpTo("login") { inclusive = true } // Hapus login dari stack juga
}

// 2. launchSingleTop - Hindari duplicate screen di top stack
navController.navigate("profile") {
    launchSingleTop = true // Jika sudah di profile, tidak buat baru
}

// 3. Kombinasi untuk login flow (setelah login, hapus login screen)
navController.navigate("home") {
    popUpTo("login") { inclusive = true }
    launchSingleTop = true
}

// 4. restoreState - Simpan dan restore state saat navigasi
navController.navigate("settings") {
    popUpTo(navController.graph.startDestinationId) { saveState = true }
    launchSingleTop = true
    restoreState = true
}
```

BAGIAN 3

PASSING ARGUMENTS

Mengirim data antar screen

REQUIRED ARGUMENTS

Mengirim data wajib ke screen tujuan:

Required Argument

```
// 1. Definisikan route dengan placeholder
sealed class Screen(val route: String) {
    object NoteDetail : Screen("note_detail/{noteId}") {
        fun createRoute(noteId: Int) = "note_detail/$noteId"
    }
}

// 2. Setup di NavHost dengan argument
composable(
    route = Screen.NoteDetail.route,
    arguments = listOf(
        navArgument("noteId") { type = NavType.IntType }
    )
) { backStackEntry ->
    val noteId = backStackEntry.arguments?.getInt("noteId") ?: 0
    NoteDetailScreen(noteId = noteId)
}

// 3. Navigasi dengan argument
navController.navigate(Screen.NoteDetail.createRoute(noteId = 42))
// URL: "note_detail/42"
```

OPTIONAL ARGUMENTS

Mengirim data opsional dengan default value:

Optional Argument

```
// 1. Definisikan route dengan query parameter (optional)
sealed class Screen(val route: String) {
    object Search : Screen("search?query={query}") {
        fun createRoute(query: String = "") =
            if (query.isEmpty()) "search" else "search?query=$query"
    }
}

// 2. Setup di NavHost dengan default value
composable(
    route = "search?query={query}",
    arguments = listOf(
        navArgument("query") {
            type = NavType.StringType
            defaultValue = "" // Default jika tidak diberikan
            nullable = true
        }
    )
) { backStackEntry ->
    val query = backStackEntry.arguments?.getString("query") ?: ""
    SearchScreen(initialQuery = query)
}

// 3. Navigasi dengan atau tanpa argument
navController.navigate(Screen.Search.createRoute()) // search
navController.navigate(Screen.Search.createRoute("kotlin")) // search?query=kotlin
```

TIPE-TIPE ARGUMENT

`NavType.IntType`

Integer (angka bulat)

```
noteId = 42
```

`NavType.LongType`

Long (angka besar)

```
timestamp = 16999999999L
```

`NavType.FloatType`

Float (desimal)

```
price = 99.99f
```

`NavType.BoolType`

Boolean (true/false)

```
isEditing = true
```

`NavType.StringType`

String (teks)

```
title = "Hello"
```



Untuk object kompleks: Serialize ke JSON string atau gunakan shared ViewModel

BAGIAN 4

NESTED NAVIGATION

Bottom Navigation dan Navigation Drawer

BOTTOM NAVIGATION

BottomNavigation.kt

```
// 1. Definisikan Bottom Nav Items
sealed class BottomNavItem(
    val route: String,
    val icon: ImageVector,
    val label: String
) {
    object Home : BottomNavItem("home", Icons.Default.Home, "Home")
    object Search : BottomNavItem("search", Icons.Default.Search, "Search")
    object Profile : BottomNavItem("profile", Icons.Default.Person, "Profile")
}

// 2. Buat Bottom Navigation Bar
@Composable
fun BottomNav(navController: NavController) {
    val items = listOf(
        BottomNavItem.Home,
        BottomNavItem.Search,
        BottomNavItem.Profile
    )

    NavigationBar {
        val currentRoute = navController
            .currentBackStackEntryAsState().value?.destination?.route

        items.forEach { item ->
            NavigationBarItem(
                selected = currentRoute == item.route,
                onClick = {
                    navController.navigate(item.route) {
                        popUpTo(navController.graph.startDestinationId)
                        launchSingleTop = true
                    }
                },
                icon = { Icon(item.icon, item.label) },
                label = { Text(item.label) }
            )
        }
    }
}
```

Preview:

Content Area



SCAFFOLD DENGAN BOTTOM NAV

MainScreen.kt

```
@Composable
fun MainScreen() {
    val navController = rememberNavController()

    Scaffold(
        bottomBar = {
            BottomNav(navController = navController)
        }
    ) { paddingValues ->
        NavHost(
            navController = navController,
            startDestination = BottomNavItem.Home.route,
            modifier = Modifier.padding(paddingValues)
        ) {
            // Bottom nav destinations
            composable(BottomNavItem.Home.route) {
                HomeScreen(
                    onItemClick = { id ->
                        navController.navigate("detail/$id")
                    }
                )
            }
            composable(BottomNavItem.Search.route) { SearchScreen() }
            composable(BottomNavItem.Profile.route) { ProfileScreen() }

            // Non-bottom nav destinations (detail screens)
            composable("detail/{id}") { backStackEntry ->
                val id = backStackEntry.arguments?.getString("id")
                DetailScreen(id = id)
            }
        }
    }
}
```

NAVIGATION DRAWER

Drawer.kt

```
@Composable
fun AppWithDrawer() {
    val navController = rememberNavController()
    val drawerState = rememberDrawerState(DrawerValue.Closed)
    val scope = rememberCoroutineScope()

    ModalNavigationDrawer(
        drawerState = drawerState,
        drawerContent = {
            ModalDrawerSheet {
                Text("Menu", modifier = Modifier.padding(16.dp))
                Divider()

                NavigationDrawerItem(
                    label = { Text("Home") },
                    selected = false,
                    onClick = {
                        navController.navigate("home")
                        scope.launch { drawerState.close() }
                    }
                )
                NavigationDrawerItem(
                    label = { Text("Settings") },
                    selected = false,
                    onClick = {
                        navController.navigate("settings")
                        scope.launch { drawerState.close() }
                    }
                )
            }
        }
    ) {
        // Main content with NavHost
        NavHost(navController, startDestination = "home") {
            composable("home") { HomeScreen() }
            composable("settings") { SettingsScreen() }
        }
    }
}
```

Preview:

Menu

 Home

 Settings

 Profile

Content

BAGIAN 5

HANDS-ON PRACTICE

3 Latihan Praktik

LATIHAN 1: NAVIGASI DASAR

Buat navigasi sederhana: Home → Detail → Back

Template Code

```
@Composable
fun AppNavigation() {
    val navController = rememberNavController()

    NavHost(
        navController = navController,
        startDestination = "home"
    ) {
        composable("home") {
            // TODO: HomeScreen dengan button navigasi
        }

        composable("detail") {
            // TODO: DetailScreen dengan back button
        }
    }
}

// TODO: Buat HomeScreen dan DetailScreen
```

Checklist:

- ☐ Setup NavController
- ☐ Setup NavHost
- ☐ HomeScreen dengan Button
- ☐ navigate() ke detail
- ☐ DetailScreen dengan Button
- ☐ popBackStack() ke home
- ☐ Test back button sistem

 Waktu: 15 menit

LATIHAN 2: NAVIGASI DENGAN ARGUMENT

Buat Note List → Note Detail dengan passing noteld

Template Code

```
// Routes dengan argument
sealed class Screen(val route: String) {
    object NoteList : Screen("note_list")
    object NoteDetail : Screen("note_detail/{noteId}") {
        fun createRoute(noteId: Int) = "note_detail/$noteId"
    }
}

// NavHost setup
composable(
    route = Screen.NoteDetail.route,
    arguments = listOf(
        navArgument("noteId") { type = NavType.IntType }
    )
) { backStackEntry ->
    val noteId = backStackEntry.arguments?.getInt("noteId") ?: 0
    // TODO: NoteDetailScreen(noteId)
}

// TODO: Implement NoteListScreen dan NoteDetailScreen
```

Checklist:

- ☐ Sealed class untuk routes
- ☐ Route dengan {noteld}
- ☐ navArgument setup
- ☐ NoteListScreen dengan clickable list items
- ☐ navigate dengan noteld
- ☐ NoteDetailScreen menampilkan noteld
- ☐ Back navigation



Waktu: 20 menit

LATIHAN 3: BOTTOM NAVIGATION

Implementasi Bottom Navigation dengan 3 tabs

Template Code

```
// Bottom Nav Items
sealed class BottomNavItem(
    val route: String,
    val icon: ImageVector,
    val label: String
) {
    object Home : BottomNavItem("home", Icons.Default.Home, "Home")
    object Favorites : BottomNavItem("favorites", Icons.Default.Favorite, "Favorites")
    object Profile : BottomNavItem("profile", Icons.Default.Person, "Profile")
}

@Composable
fun MainScreen() {
    val navController = rememberNavController()

    Scaffold(
        bottomBar = {
            // TODO: Implement NavigationBar
        }
    ) { padding ->
        NavHost(
            navController = navController,
            startDestination = BottomNavItem.Home.route,
            modifier = Modifier.padding(padding)
        ) {
            // TODO: Add composable destinations
        }
    }
}
```

Checklist:

- ☐ BottomNavItem sealed class
- ☐ NavigationBar component
- ☐ NavigationBarItem untuk setiap tab
- ☐ currentBackStackEntry untuk selected state
- ☐ Scaffold dengan bottomBar
- ☐ NavHost di content
- ☐ 3 screen composables
- ☐ Test tab switching

 Waktu: 25 menit

TROUBLESHOOTING

✗ NavController not found

Penyebab: NavController belum dibuat atau tidak diteruskan

✓ Solusi: Gunakan `rememberNavController()` dan pass ke composable

✗ Destination not found

Penyebab: Route tidak terdaftar di NavHost

✓ Solusi: Pastikan semua route sudah didefinisikan di NavHost

✗ Argument mismatch

Penyebab: Tipe argument tidak sesuai

✓ Solusi: Periksa NavType dan pastikan sesuai dengan data yang dikirim

✗ Back stack issue

Penyebab: `popUpTo` salah konfigurasi

✓ Solusi: Periksa `inclusive` parameter dan route name

BEST PRACTICES

✓ Centralized Routes

Definisikan semua routes di satu tempat (sealed class)

✓ Type-safe Arguments

Gunakan sealed class dengan helper function untuk arguments

✓ Single NavHost

Gunakan satu NavHost di root, nested navigation jika perlu

✓ Avoid NavController in ViewModel

Gunakan callback/events, jangan pass NavController ke ViewModel

✓ Handle Deep Links

Definisikan deep link patterns untuk setiap destination

✓ Test Navigation

Unit test navigation logic dengan TestNavController

RANGKUMAN PERTEMUAN 5

Navigation Component

NavHost, NavController, Routes

Basic Navigation

navigate() dan popBackStack()

Arguments

Required dan Optional arguments dengan NavType

Navigation Options

popUpTo, launchSingleTop, restoreState

Bottom Navigation

NavigationBar dengan NavigationBarItem

Navigation Drawer

ModalNavigationDrawer untuk side menu

 **Navigation = User journey through your app!**

TUGAS PRAKTIKUM MINGGU 5

Bobot: 4% | Deadline: Sebelum Pertemuan 6



Deskripsi Tugas:

Kembangkan Notes App dari minggu lalu dengan fitur navigasi:

1. Bottom Navigation dengan 3 tabs: Notes, Favorites, Profile
2. Note List → Note Detail navigation dengan passing noteld
3. Floating Action Button untuk Add Note (navigate ke AddNoteScreen)
4. Back navigation yang proper dari semua screens
5. Edit Note screen dengan passing noteld sebagai argument



Format Pengumpulan:

- Push ke GitHub repository (branch: week-5)
- README: Screenshot setiap screen dan navigation flow diagram
- Struktur folder: navigation/, screens/, components/
- Video demo (30 detik) menunjukkan semua navigation flows

RUBRIK PENILAIAN TUGAS

Komponen	Bobot	Kriteria
Bottom Navigation	25%	3 tabs berfungsi dengan state yang benar
Navigation with Args	25%	notelId passing dan display benar
Navigation Flow	20%	Forward dan back navigation proper
Code Structure	20%	Routes terorganisir, clean code
Documentation	10%	README, screenshots, video demo

🌟 Bonus (+10%)

Navigation Drawer tambahan

⚠️ Penalti

Terlambat: -10%/hari, Plagiat: 0

SUMBER PUSTAKA

Dokumentasi

Navigation in Compose

developer.android.com/jetpack/compose/navigation

Dokumentasi

Navigate with arguments

developer.android.com/guide/navigation/navigation-pass-data

Guide

Bottom Navigation in Compose

developer.android.com/jetpack/compose/navigation#bottom-nav

Library

Voyager (KMP Navigation)

github.com/adrielcafe/voyager

Library

Decompose (KMP Navigation)

github.com/arkivanov/Decompose

Codelab

Navigation in Jetpack Compose

developer.android.com/codelabs/jetpack-compose-navigation

PREVIEW MINGGU DEPAN

Pertemuan 6: Networking dan REST API

HTTP dan REST API

Konsep dasar web services

Ktor Client

HTTP client untuk KMP

JSON Serialization

Parsing JSON response

Loading States

Handle loading dan error

Persiapan:

- Selesaikan tugas Notes App dengan navigasi
- Baca dokumentasi Ktor Client
- Review konsep HTTP methods (GET, POST, PUT, DELETE)



SESI TANYA JAWAB

Ada pertanyaan tentang Navigation di Compose?

CHECKLIST SEBELUM PULANG

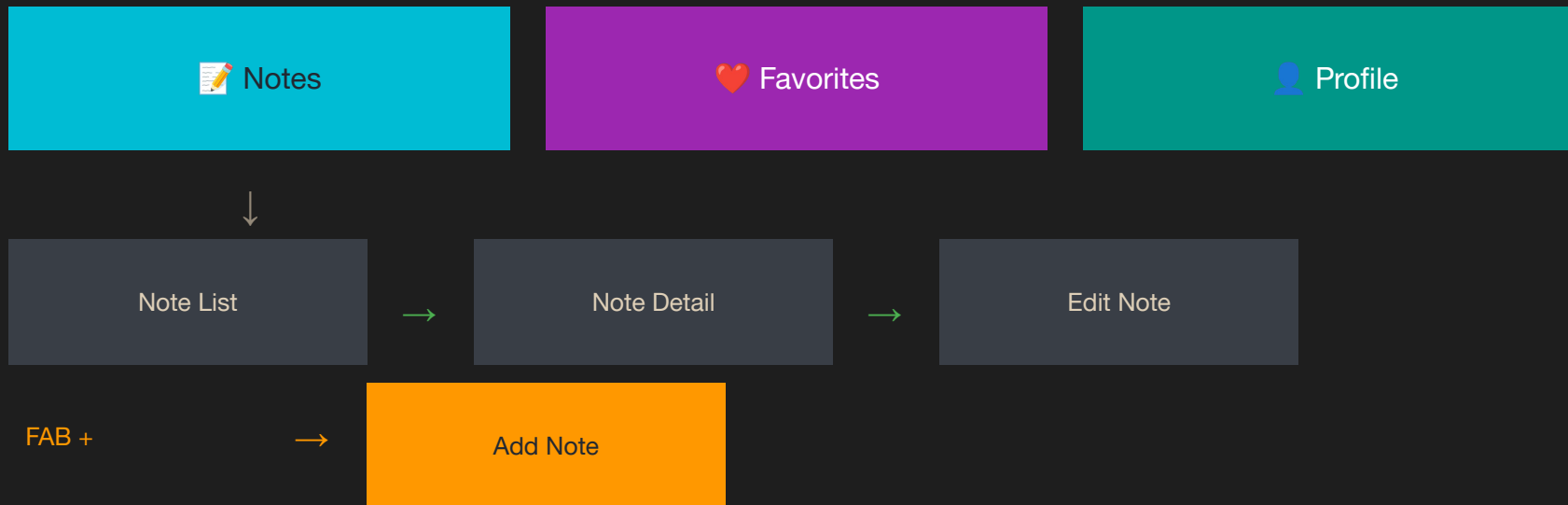
Pastikan Anda sudah:

- ☐ Memahami konsep navigasi dalam aplikasi mobile
- ☐ Bisa setup NavHost dan NavController
- ☐ Mengerti cara definisi routes dengan sealed class
- ☐ Bisa navigasi dengan navigate() dan popBackStack()
- ☐ Mampu passing arguments antar screen
- ☐ Memahami implementasi Bottom Navigation
- ☐ Tahu cara menggunakan Scaffold dengan bottomBar
- ☐ Mencatat deadline tugas Notes App dengan navigasi

CONTOH NAVIGATION FLOW

Notes App Navigation Architecture:

Bottom Navigation Tabs:



Legend:

→ navigate() ← popBackStack() Tab switch: popUpTo + launchSingleTop

Arguments: `notelId (Int)` passed to Detail and Edit screens

ALTERNATIF: LIBRARY NAVIGASI KMP

Selain Navigation Compose, ada beberapa library populer untuk KMP:

Voyager

- Pragmatic navigation library
- Simple API, easy to learn
- Screen-based navigation
- Tab navigation support
- BottomSheet navigation

Decompose

- Component-based architecture
- Lifecycle-aware
- State preservation
- Deep linking support
- More complex but powerful

PreCompose

- Jetpack-like API
- ViewModel support
- Navigation similar to Jetpack
- Easy migration from Android
- Active development



Rekomendasi

Untuk project ini, kita gunakan:

- Navigation Compose (official)
- Atau Voyager (lebih simple)

Pilih berdasarkan kebutuhan dan kompleksitas project!

Terima Kasih

Pertemuan 5: Navigasi Antar Layar

"Good navigation is invisible. Users should focus on content, not on how to get there."

- UX Design Principle

Selamat membangun aplikasi dengan navigasi yang intuitif! 🧭